

Instituto Tecnológico Superior de Río verde SLP.

TEMA:

Semana 12

ALUMNO:

Brayan Ivanov Martínez Martínez

MATERIA:

programacion

DOCENTE:

Jesus collazo

GRUPO

2ª A

Fecha

7/05/2026

Introducción

El polimorfismo es una característica muy importante de la Programación Orientada a Objetos (POO). Este permite que diferentes objetos puedan realizar una misma acción de distintas maneras. Gracias a esto, los programas pueden manejar varios tipos de objetos usando los mismos métodos, haciendo el código más organizado y fácil de reutilizar.

En los videojuegos, el polimorfismo ayuda mucho porque permite crear personajes con habilidades diferentes sin tener que hacer todo el código desde cero para cada uno. Por ejemplo, pueden existir personajes como magos, guerreros o arqueros, donde cada uno ataca o se defiende de manera distinta, pero todos siguen siendo parte del mismo tipo de personaje dentro del juego.

En esta práctica se realizó un pequeño sistema de combate en C++ utilizando herencia, funciones virtuales y polimorfismo. Se creó una clase principal llamada Personaje y varias clases derivadas con diferentes comportamientos y habilidades. También se agregó un menú interactivo para que el usuario pudiera probar el funcionamiento del programa.

El objetivo principal de esta práctica fue entender cómo el polimorfismo ayuda a crear programas más flexibles, ordenados y fáciles de modificar, además de conocer cómo se aplican estos conceptos en videojuegos y programas reales.

Objetivo General

Aplicar el polimorfismo en C++ utilizando funciones virtuales, clases abstractas y punteros a clases base, para que diferentes personajes puedan ejecutar una misma función de distintas maneras según su tipo.

Desarrollo de la práctica

En esta práctica se realizó un sistema de combate basado en videojuegos tipo RPG.

El programa fue organizado en tres archivos principales:

- * Personaje.h
- * Personaje.cpp
- * main.cpp

La clase principal llamada `Personaje` contiene los datos básicos que comparten todos los personajes, como el nombre, la vida y el nivel. También se creó un método virtual puro llamado `atacar()`, haciendo que la clase funcione como una clase abstracta.

Después se crearon tres clases derivadas:

- * Guerrero
- * Mago
- * Arquero

Cada una tiene su propia forma de atacar, por lo que el método `atacar()` funciona diferente dependiendo del personaje seleccionado.

se utilizó un vector de punteros de tipo `Personaje` para guardar distintos personajes dentro de una misma lista. Aquí es donde se aplica el polimorfismo, ya que el programa puede trabajar con todos los personajes usando la misma referencia.

Por último se agregó un menú interactivo para que el usuario pudiera realizar diferentes acciones como:

- * Ver la información de los personajes
- * Ejecutar ataques
- * Simular combates entre personajes

```

semana12 > C personaje.h
1  #ifndef PERSONAJE_H
2  #define PERSONAJE_H
3
4  #include <iostream>
5  #include <string>
6
7  using namespace std;
8
9  class Personaje {
10 protected:
11     string nombre;
12     int nivel;
13     int vida;
14
15 public:
16     Personaje(string nom, int niv, int vid);
17     virtual ~Personaje();
18
19     void mostrarInfo();
20     virtual void atacar() = 0;
21
22     int recibirDanio(int cantidad);
23     bool estaVivo();
24
25     string getNombre();
26     int getVida();
27 };
28
29 class Guerrero : public Personaje {
30 public:
31     Guerrero(string nom, int niv, int vid);
32     void atacar() override;
33 };
34
35 class Mago : public Personaje {
36 public:
37     Mago(string nom, int niv, int vid);

```

```

36 public:
37     Mago(string nom, int niv, int vid);
38     void atacar() override;
39 };
40
41 class Arquero : public Personaje {
42 public:
43     Arquero(string nom, int niv, int vid);
44     void atacar() override;
45 };
46
47 // combate
48 void combatir(Personaje* p1, Personaje* p2);
49
50 #endif

```

El archivo header nos ayuda a definir las clases y los objetos del tipo `Personaje` además de establecer sus atributos y métodos. También permite agregar los diferentes comportamientos de los personajes como mostrar información, recibir daño o realizar ataques mediante funciones como `mostrarInfo()`, `recibirDanio()` y `atacar()`.

```

semana12 > g++ personaje.cpp
1  Personaje::Personaje(string nom, int niv, int vid) {
2      nombre = nom;
3      nivel = niv;
4      vida = vid;
5  }
6
7  Personaje::~Personaje() {
8  }
9
10 void Personaje::mostrarInfo() {
11     cout << "Nombre: " << nombre << endl;
12     cout << "Nivel: " << nivel << endl;
13     cout << "Vida: " << vida << endl;
14 }
15
16 void Personaje::recibirDanio(int cantidad) {
17     vida -= cantidad;
18
19     if (vida < 0) {
20         vida = 0;
21     }
22 }
23
24 bool Personaje::estaVivo() {
25     return vida > 0;
26 }
27
28 string Personaje::getNombre() {
29     return nombre;
30 }
31
32 int Personaje::getVida() {
33     return vida;
34 }
35
36 // guerrero
37 Guerrero::Guerrero(string nom, int niv, int vid)

```

```

semana12 > g++ personaje.cpp
30 // guerrero
37 Guerrero::Guerrero(string nom, int niv, int vid)
38     : Personaje(nom, niv, vid) {
39 }
40
41 void Guerrero::atacar() {
42     cout << nombre << " realiza un ataque poderoso con espada " << endl;
43 }
44
45 // mago
46 Mago::Mago(string nom, int niv, int vid)
47     : Personaje(nom, niv, vid) {
48 }
49
50 void Mago::atacar() {
51     cout << nombre << " lanza una bola de fuego " << endl;
52 }
53
54 // arquero
55 Arquero::Arquero(string nom, int niv, int vid)
56     : Personaje(nom, niv, vid) {
57 }
58
59 void Arquero::atacar() {
60     cout << nombre << " dispara una flecha al objetivo " << endl;
61 }
62
63 // funcion de combate
64 void combatir(Personaje* p1, Personaje* p2) {
65
66     if (!p1->estaVivo() || !p2->estaVivo()) {
67         cout << "\nUno de los personajes ya no puede combatir.\n";
68         return;
69     }
70
71     cout << "\n=== COMBATE ===\n";
72

```

En esta parte del código se implementan y desarrollan los métodos y funciones que serán utilizados en el `main`.

Aquí se define el funcionamiento de cada acción del programa indicando qué hará cada función y cómo responderá en diferentes situaciones. También se implementan métodos como `getVida()`, que permite obtener la vida actual del personaje además de las acciones de ataque, recibir daño y el sistema de combate entre personajes.

```

76
77     p1->atacar();
78
79     int dano = 20;
80
81     p2->recibirDanio(dano);
82
83     cout << p2->getNombre()
84         << " recibió "
85         << dano
86         << " puntos de daño.\n";
87
88     cout << "Vida restante de "
89         << p2->getNombre()
90         << ": "
91         << p2->getVida()
92         << endl;
93
94     if (!p2->estaVivo()) {
95         cout << p2->getNombre()
96             << " ha sido derrotado \n";
97     }
98 }

```

```

semana12 > main.cpp
1  vector<Personaje*> personajes;
2
3  // Crear personajes
4  personajes.push_back(new Guerrero("Leila", 10, 100));
5  personajes.push_back(new Mago("Mario", 8, 80));
6  personajes.push_back(new Arquero("jose", 9, 90));
7
8  int opcion;
9
10 do {
11     cout << "\n=== MENU DEL JUEGO ===\n";
12     cout << "1. Mostrar personajes\n";
13     cout << "2. Ejecutar ataques\n";
14     cout << "3. Combatir\n";
15     cout << "4. Salir\n";
16     cout << "Seleccione una opcion: ";
17     cin >> opcion;
18
19     switch(opcion) {
20
21     case 1:
22
23         cout << "\n=== INFORMACION ===\n";
24
25         for (int i = 0; i < personajes.size(); i++) {
26
27             cout << "\nPersonaje #" << i + 1 << endl;
28
29             personajes[i]->mostrarInfo();
30
31         }
32
33         break;
34
35     case 2:
36
37         cout << "\n=== ATAQUES ===\n";

```

```

semana12 > main.cpp
10  do {
34      case 2:
38          for (Personaje* p : personajes) {
39              p->atacar();
40          }
41
42          break;
43
44      case 3:
45
46          int atacante, objetivo;
47
48          cout << "\nSeleccione atacante:\n";
49
50          for (int i = 0; i < personajes.size(); i++) {
51
52              cout << i + 1 << ". "
53                  << personajes[i]->getNombre()
54                  << endl;
55          }
56
57          cin >> atacante;
58
59          cout << "\nSeleccione objetivo:\n";
60
61          for (int i = 0; i < personajes.size(); i++) {
62
63              cout << i + 1 << ". "
64                  << personajes[i]->getNombre()
65                  << endl;
66          }
67
68          cin >> objetivo;
69
70          if (atacante >= 1 && atacante <= personajes.size() &&
71              objetivo >= 1 && objetivo <= personajes.size()) {
72
73              combatir(
74                  personajes[atacante - 1],
75                  personajes[objetivo - 1]
76              );
77
78          } else {
79
80              cout << "\nOpcion invalida.\n";
81
82          }
83
84          break;
85
86      case 4:
87
88          cout << "\nSaliendo del juego...\n";
89
90          break;
91
92      default:
93
94          cout << "\nOpcion no valida.\n";
95
96      }
97  } while(opcion != 4);
98
99  // Liberar memoria
100  for (Personaje* p : personajes) {
101      delete p;
102  }
103
104  return 0;

```

El `main` es la parte principal del programa donde se crean los personajes y se muestra el menú interactivo para el usuario. Desde ahí el usuario puede elegir diferentes opciones como ver la información de los personajes, ejecutar ataques realizar combates entre ellos o salir del programa.

Además en esta parte se utilizan las funciones creadas anteriormente para demostrar cómo funcionan el polimorfismo la herencia y los diferentes métodos implementados en las clases, permitiendo que cada personaje actúe de manera distinta durante el combate.

```

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana12
$ g++ main.cpp personaje.cpp -o programa

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana12
$ ./programa

```

```

=== MENU DEL JUEGO ===
1. Mostrar personajes
2. Ejecutar ataques
3. Combatir
4. Salir
Seleccione una opcion:

```

```

=== INFORMACION ===

```

```

Personaje #1
Nombre: Leika
Nivel: 10
Vida: 100

```

```

Personaje #2
Nombre: Nilo
Nivel: 8
Vida: 80

```

```

Personaje #3
Nombre: Zoe
Nivel: 9
Vida: 90

```

```

=== ATAQUES ===

```

```

Leika realiza un ataque poderoso con espada
Nilo lanza una bola de fuego
Zoe dispara una flecha al objetivo

```

```

=== MENU DEL JUEGO ===

```

Explicación

¿Cómo se implementó el polimorfismo?

El polimorfismo se implementó usando funciones virtuales y punteros a la clase base `Personaje`. Para esto, se utilizó un vector de punteros `Personaje\*`, donde se guardan objetos de diferentes clases derivadas como `Guerrero`, `Mago` y `Arquero`. Gracias a esto, todos los personajes pueden manejarse de forma general, pero cada uno ejecuta acciones diferentes según su tipo.

¿Qué es una función virtual?

Una función virtual es un método que se declara en una clase base y que puede ser redefinido en las clases derivadas. Esto permite que el programa decida automáticamente qué versión del método ejecutar dependiendo del objeto que se esté utilizando en ese momento.

¿Qué ventaja tiene usar punteros a la clase base?

Usar punteros a la clase base ayuda a trabajar con diferentes tipos de objetos usando una misma referencia. Esto hace que el código sea más organizado, flexible y fácil de reutilizar, especialmente en programas grandes como videojuegos.

Conclusiones

Con esta práctica pude entender mejor cómo funciona el polimorfismo en C++. También reforcé temas importantes como la herencia, las clases abstractas, las funciones virtuales y el uso de punteros a clases base.

Al principio fue un poco difícil comprender cómo el programa podía ejecutar diferentes métodos usando la misma función, pero al ver que cada personaje atacaba de manera distinta con el método `atacar()`, el funcionamiento del polimorfismo quedó mucho más claro.

Además, esta práctica me ayudó a darme cuenta de que estos conceptos no solo sirven para ejercicios escolares, sino que también son muy utilizados en videojuegos y programas reales, ya que permiten crear sistemas más ordenados, reutilizables y fáciles de mantener.

1. ¿Qué problema se quiere resolver con el polimorfismo?

Se busca que varios personajes puedan hacer la misma acción sin importar qué tipo sean Así el sistema trabaja igual con todos y cada personaje actúa diferente.

2. ¿Qué clase tendría el comportamiento general de los personajes?

La clase Personaje sería la principal porque tiene lo que todos comparten como nombre vida y nivel.

3. ¿Qué método sería bueno para usar polimorfismo?

El método atacar() porque todos atacan pero cada personaje lo hace de forma distinta.

4. ¿Por qué ese método debe ser virtual?

Porque así cada clase puede cambiar la forma en que funciona el método atacar().

5. ¿Qué pasaría si el método no fuera virtual?

Todos usarían el mismo ataque de la clase base y no habría diferencias entre personajes.

6. ¿Cómo harías que cada personaje ataque diferente?

Creando clases como Guerrero, Mago y Arquero y cambiando el método atacar() en cada una.

7. ¿Cómo puede el sistema usar ataques sin saber qué personaje es?

Usando punteros o referencias de la clase Personaje para llamar el método atacar().

8. ¿Qué estructura usarías para guardar muchos personajes?

Usaría un vector de tipo Personaje* para guardar varios personajes diferentes.

9. ¿Qué ventaja tiene usar punteros o referencias a la clase base?

Permite manejar todos los personajes de la misma manera y hace el código más fácil.

10. ¿Qué significa que se decide en tiempo de ejecución?

Que el programa elige qué método usar mientras está funcionando.

11. ¿Cómo ayuda el polimorfismo a evitar condicionales?

Porque cada objeto ya sabe qué hacer y no hace falta usar muchos if o switch.

12. ¿Qué pasaría si agregas un personaje nuevo como Hechicero?

Solo habría que crear la clase Hechicero y darle su propio ataque.

13. ¿Se tendría que cambiar el sistema principal? ¿Por qué?

No, porque el sistema ya está preparado para agregar nuevos personajes.

14. ¿Qué ventajas tiene el polimorfismo en videojuegos o programas grandes?

Hace el código más ordenado, fácil de mantener y más sencillo de ampliar.

15. ¿Qué relación hay entre herencia y polimorfismo?

La herencia crea nuevos personajes y el polimorfismo permite que actúen diferente.

16. ¿Qué es una interfaz común y cómo funciona aquí?

Es algo que todos los personajes comparten. Aquí sería la clase Personaje con métodos como atacar().

17. ¿Qué problemas habría sin polimorfismo?

El código sería más largo, repetitivo y difícil de modificar.

18. ¿Cómo mejorarías el diseño para agregar más acciones?

Agregando métodos como `defender()` o `curar()` para que cada personaje los haga a su manera.

19. ¿Cuándo puede complicarse el polimorfismo?

Cuando hay demasiadas clases y el sistema se vuelve más difícil de entender.

20. ¿Cómo explicarías el polimorfismo con este ejemplo?

Es cuando varios personajes usan el mismo método `atacar()`, pero cada uno pelea diferente según su tipo.